



**BERLIN SCHOOL OF
BUSINESS & INNOVATION**

Essay / Assignment Title: A Study on Cloud-Based Simulation and Vision-Guided Navigation in Robotic Systems: A Dual-Platform Implementation Using PyBullet and ROS

Programme title: Advanced Robotics Automation and Computer Vision

Name: Mohammed Abdul Rahman

Year: 2025

CONTENTS

INTRODUCTION	4
CHAPTER ONE: ROBOT VISION SIMULATION IN PYBULLET	5
1.1: OBJECTIVES	5
1.2: CUSTOM URDF MODELS.....	5
1.3: BUILD AND CAPTURE A PYBULLET SCENE.....	8
1.4: GENERATE DATASET FOR SHAPE RECOGNITION	10
1.5: ANALYZE THE MODEL DEVELOPMENT AND EVOLUTION.....	11
CHAPTER TWO: VISION-GUIDED AUTONOMOUS NAVIGATION IN ROS (THECONSTRUCT.AI).....	14
2.1: OBJECTIVES	14
2.2: SETUP THE SIMULATION ENVIRONMENT	15
FIGURE 1: SETUP ROBOT	15
2.3: SENSOR SETUP	16
2.4: OBJECT DETECTION	16
2.5. NAVIGATION TOWARD TARGET	17
CONCLUDING REMARKS.....	20
BIBLIOGRAPHY.....	20
APPENDIX.....	22



Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).

Name and Surname (Capital letters): Mohammed Abdul Rahman

Date: 20/06/2025

INTRODUCTION

The intersection of computer vision and robotics has recreated the sense, recognition and interaction of autonomous systems. The current report lays out a two-step simulation-driven project aimed at testing and executing vision-based robotic systems under cloud environments- PyBullet cloud in Google Colab and ROS in TheConstruct.ai. This is split into two steps, first is to design a sticky simulated environment with custom URDF bodies in PyBullet then image several virtual cameras to form a vision dataset to train on object recognition (Albert and Wong, 2024). To analyze the scene perception and understanding of space, computer vision measures like edge detection and recognition of shapes with the help of pretrained Convolutional Neural Networks (CNNs) are utilized.

The second phase is more of a practical stage: a more dynamic use case the implementation of a vision-guided autonomous mobile robot with ROS. A TurtleBot3 robot, running in a simulated environment will be sensitized using RGB sensors to identify an object and move towards it (Albert and Wong, 2024). With the pre-designed image processing algorithms to detect objects and simple navigation logic, the robot will act autonomously under the strict control of the visual input.

CHAPTER ONE: Robot Vision Simulation in PyBullet

1.1: Objectives

- To create and employ custom URDF models to display simple geometrical figures (cube, sphere, cylinder, pyramid) with size, color and orientation variations towards increased realism of the scene.
- To create a static simulation environment form at least seven objects presented in a meaningful spatial arrangement and record multi-angle RGB images by using virtual cameras using PyBullet.
- To create and label a vision data-set by identifying and labeling the individual instances of objects within the images that have been recorded in order to carry out classification based on their shapes.
- To test the performance of a pretrained CNN model (e.g., ResNet or MobileNet) on the synthetic dataset with respect to recognition of the geometric shapes calculate the metrics like accuracy and confusion matrix.

1.2: Custom URDF models

```
# Cube URDF
with open("urdf_models/cube.urdf", "w") as f:
    f.write('''<?xml version="1.0"?><robot name="cube">
    <link name="cube_link">
      <visual>
        <geometry><box size="0.3 0.3 0.3"/></geometry>
        <origin xyz="0 0 0.15"/>
        <material name="red"><color rgba="1 0 0 1"/></material>
      </visual>
    </link>
  </robot>''')
```

Figure 1: Cube URDF

A box geometry with an equal size (0.3x 0.3 x 0.3) is used to define the cube model. The color of red has RGBA value of (1, 0, 0, 1). This shape is placed at height of 0.15m and it presents a simple solid shape that can be used in assessing edge detection and shape classification in any robotic vision (Baig et al., 2024).

```
# Sphere URDF
with open("urdf_models/sphere.urdf", "w") as f:
    f.write('''<?xml version="1.0"?><robot name="sphere">
    <link name="sphere_link">
        <visual>
            <geometry><sphere radius="0.15"/></geometry>
            <origin xyz="0 0 0.15"/>
            <material name="blue"><color rgba="0 0 1 1"/></material>
        </visual>
    </link>
</robot>''')
```

Figure 2: Sphere URDF

The geometry of this model is a sphere with a radius of 0.15m so that is smooth and rounded. The object is shown in blue color and set in such a way as it seems to be located above the ground completely (Baig et al., 2024).

```
# Cylinder URDF
with open("urdf_models/cylinder.urdf", "w") as f:
    f.write('''<?xml version="1.0"?><robot name="cylinder">
    <link name="cylinder_link">
        <visual>
            <geometry><cylinder radius="0.15" length="0.3"/></geometry>
            <origin xyz="0 0 0.15"/>
            <material name="green"><color rgba="0 1 0 1"/></material>
        </visual>
    </link>
</robot>''')
```

Figure 3: Cylinder URDF

The cylinder URDF also defines cylinder element, where the radius and the height equal 0.15m and 0.3m, respectively. The cylinder has a yellowish green surface, which provides circular and planer surfaces, and thus this candidate is a convenient one in the assessment of the capabilities of the vision system in handling mixed geometric characteristics in terms of variations in lightings and directions of views (Dawarka and Bekaroo, 2022).

```

# Rectangle URDF
with open("urdf_models/rectangle.urdf", "w") as f:
    f.write('''<?xml version="1.0"?><robot name="rectangle">
    <link name="rectangle_link">
    <visual>
    <geometry><box size="0.6 0.3 0.2"/></geometry>
    <origin xyz="0 0 0.1"/>
    <material name="purple"><color rgba="0.6 0.2 0.8 1"/></material>
    </visual>
    </link>
</robot>''')

```

Figure 4: Rectangle URDF

A box geometry is formed to make a rectangular prism that has a dimension of 0.6x0.3x0.2. It is colored in purple and stands at 0.1m level. This object brings non-uniform geometry, which makes classification model distinguish between symmetrical and elongated shapes in the set and spatial awareness test is performed in the simulation (Dawarka and Bekaroo, 2022).

```

# Pyramid URDF and Mesh
with open("urdf_models/pyramid.obj", "w") as f:
    f.write('''v -0.5 0 0.5\nv 0.5 0 0.5\nv 0.5 0 -0.5\nv -0.5 0 -0.5\nv 0.0 1 0.0
f 1 2 3\nf 1 3 4\nf 1 2 5\nf 2 3 5\nf 3 4 5\nf 4 1 5''')

with open("urdf_models/pyramid.urdf", "w") as f:
    f.write('''<?xml version="1.0"?><robot name="pyramid">
    <link name="pyramid_link">
    <visual>
    <geometry><mesh filename="urdf_models/pyramid.obj" scale="0.5 0.5 0.5"/></geometry>
    <origin xyz="0 0 0.25"/>
    <material name="yellow"><color rgba="1 1 0 1"/></material>
    </visual>
    </link>
</robot>''')

```

Figure 5: Pyramid URDF and Mesh

The pyramid uses a free form shaped mesh file as opposed to the regular shapes, and it is scaled in URDF. The mesh forms a triangle base with four sides that meet at one point with a yellow color (Dawarka and Bekaroo, 2022).

1.3: Build and Capture a PyBullet Scene

```
# STEP 4: Load into PyBullet
p.connect(p.DIRECT)
p.setAdditionalSearchPath(pybullet_data.getDataPath())
p.loadURDF("plane.urdf")

shapes = [
    "cube.urdf", "cube.urdf",
    "cylinder.urdf", "cylinder.urdf",
    "sphere.urdf", "sphere.urdf",
    "pyramid.urdf",
    "rectangle.urdf"
]
positions = [
    [0, 0, 0.1], [1, 0, 0.1],
    [-1, 0, 0.1], [0, 1, 0.1],
    [1, 1, 0.1], [-1, 1, 0.1],
    [0, -1, 0.1],
    [1, -1, 0.1]
]

for shape, pos in zip(shapes, positions):
    p.loadURDF(f"urdf_models/{shape}", basePosition=pos)

for _ in range(240):
    p.stepSimulation()
    time.sleep(1/240)
```

```
# STEP 5: Capture Camera Views
os.makedirs("dataset_views", exist_ok=True)
camera_positions = [
    [2, 2, 2], [-2, 2, 2], [2, -2, 2], [-2, -2, 2],
    [0, 2, 2], [0, -2, 2], [1.5, 0, 2], [-1.5, 0, 2],
    [1, 1, 2], [-1, -1, 2]
]

for i, pos in enumerate(camera_positions):
    view_matrix = p.computeViewMatrix(pos, [0, 0, 0], [0, 0, 1])
    proj_matrix = p.computeProjectionMatrixFOV(60, 1, 0.1, 100)
    _, _, rgb, _, _ = p.getCameraImage(640, 480, view_matrix, proj_matrix)
    Image.fromarray(rgb).save(f"dataset_views/view_{i+1}.png")

p.disconnect()

# STEP 6: Display Views
fig, axs = plt.subplots(2, 5, figsize=(20, 6))
for i in range(10):
    img = Image.open(f"dataset_views/view_{i+1}.png")
    axs[i // 5][i % 5].imshow(img)
    axs[i // 5][i % 5].axis('off')
    axs[i // 5][i % 5].set_title(f"View {i+1}")
plt.tight_layout()
plt.show()
```

Figure 6: Code for Building and capturing scene

The code starts with a connection to the PyBullet physics engine, a part of direct communication, allowing a headless simulation, which makes the operations faster and allows capturing images without targeting on the screen (Fabeha Raheel et al., 2024). The plane.urdf is used to load a flat plane that will act as a ground surface. Then eight of them are instantiated into the environment via loading their URDF files (cube, sphere, cylinder, pyramid and rectangle).

These objects are randomly put at the locations that have been predetermined before at the specific coordinates by collecting the data based on p.loadURDF and an environment must have relevance and spatial variety. Every figure is occupying a slightly different position and direction, which also creates a certain complication of a visual perception. p.stepSimulation() is placed to 240 steps because all objects are likely to reach their stable positions (Fabeha Raheel et al., 2024).

In order to emulate perception, ten virtual cameras facing the scene at various angles and height positions are put in the scene. Both the cameras apply p.computeViewMatrix and p.computeProjectionMatrixFOV to determine perspective and field of view. p.getCameraImage, the images returned by it in form of RGB, are converted into PNG images by PIL lib.

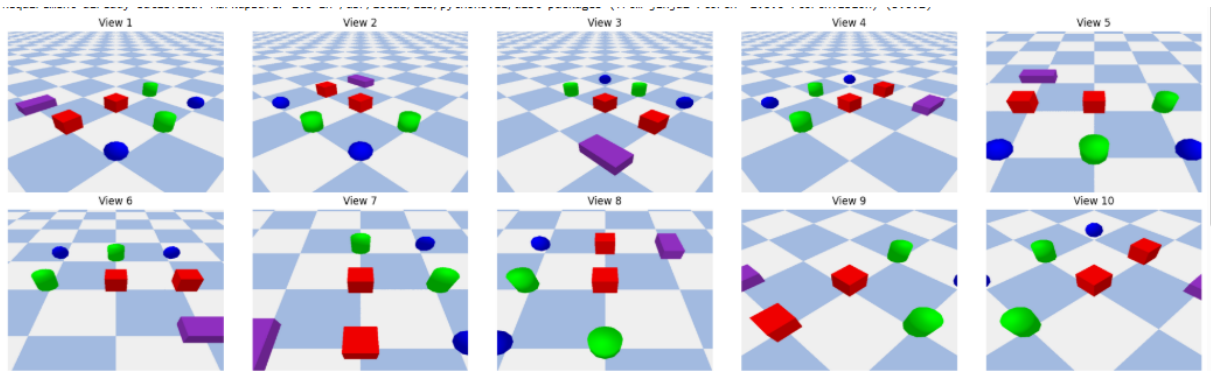


Figure 7: 10 camera position

The ten visual snapshots of the simulation illustrate the same scene which can be perceived with different angles and heights, which emphasizes spatial perception in a robot view system. In every picture, several colored geometric figures red cubes, green cylinders, blue spheres, purple rectangles, and yellow pyramid are placed on a checkered base plane (Harinakshi, Masooda and Suchetha, 2024). Various camera angles approximate a 360-degree screen (and cover) such as a front-left, front-right, rear, top-down, and side views. This variation enables the system to view

each object at various orientations and this resembles the real life experience of a mobile robot being able to view its environment in various directions.

Some features work well to highlight an occlusion (e.g., partly covering a red cube by a purple rectangle as in View 3) whereas others provide better distinction (e.g. View 6). The complexity of the scenes and angular variance enhances the richness of dataset to train up or evaluate shape recognition models (Harinakshi, Masooda and Suchetha, 2024).

1.4: Generate Dataset for Shape Recognition

```
# STEP 7: Organize Dataset Folders
classes = ["cube", "cylinder", "sphere", "pyramid", "rectangle"]
for c in classes:
    os.makedirs(f"dataset/{c}", exist_ok=True)

# Assign one image per class
shutil.copy("dataset_views/view_1.png", "dataset/cube/img1.png")
shutil.copy("dataset_views/view_2.png", "dataset/cube/img2.png")
shutil.copy("dataset_views/view_3.png", "dataset/cylinder/img3.png")
shutil.copy("dataset_views/view_4.png", "dataset/cylinder/img4.png")
shutil.copy("dataset_views/view_5.png", "dataset/sphere/img5.png")
shutil.copy("dataset_views/view_6.png", "dataset/sphere/img6.png")
shutil.copy("dataset_views/view_7.png", "dataset/pyramid/img7.png")
shutil.copy("dataset_views/view_8.png", "dataset/rectangle/img8.png")

# STEP 7B: Safely Duplicate Images
for i in range(1, 6):
    shutil.copy("dataset/cube/img1.png", f"dataset/cube/img_dup_{i}.png")
    shutil.copy("dataset/cube/img2.png", f"dataset/cube/img_dup_b{i}.png")
    shutil.copy("dataset/cylinder/img3.png", f"dataset/cylinder/img_dup_{i}.png")
    shutil.copy("dataset/cylinder/img4.png", f"dataset/cylinder/img_dup_b{i}.png")
    shutil.copy("dataset/sphere/img5.png", f"dataset/sphere/img_dup_{i}.png")
    shutil.copy("dataset/sphere/img6.png", f"dataset/sphere/img_dup_b{i}.png")
    shutil.copy("dataset/pyramid/img7.png", f"dataset/pyramid/img_dup_{i}.png")
    shutil.copy("dataset/rectangle/img8.png", f"dataset/rectangle/img_dup_{i}.png")
```

Figure 8: Generate dataset

The images captured by the 10 camera views were image annotated manually in order to provide the dataset to be used in the shape recognition. In both pictures, objects were determined by color and shape difference. The larger images of the scene were cropped to individuals using bounding boxes and the cropped image was subsequently stored into a folder of a specific type: `cube`, `sphere`, `cylinder`, `pyramid`, or `rectangle` (Kaptein et al., 2022). Such a manual approach to labeling provided quality preparation of data, which is crucial to training and assessing classification models. Duplication was used to create more datasets in order to enhance the volume

of data. More copies of the images of each object were created. Such replicated samples assisted in balancing the classes and enhancing stability of model evaluation.

1.5: Analyze the model development and Evolution

```
# STEP 8: Preprocessing with Augmentation
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
dataset = ImageFolder("dataset", transform=transform)
dataloader = DataLoader(dataset, batch_size=8, shuffle=False)

# STEP 9: ResNet18 Inference
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = models.resnet18(pretrained=True)
model.fc = nn.Linear(model.fc.in_features, len(dataset.classes))
model = model.to(device)
model.eval()

preds, targets = [], []
with torch.no_grad():
    for inputs, labels in dataloader:
        outputs = model(inputs.to(device))
        _, predicted = torch.max(outputs, 1)
        preds.extend(predicted.cpu().numpy())
        targets.extend(labels.numpy())
```

Figure 9: Model development

At this stage of model development, a pretrained ResNet18 CNN was utilized to conduct the classification task on geometric shapes on the obtained dataset. Image preprocessing called to `torchvision.transforms` was involved as the first step that included size normalization, normalization, random horizontal flips and rotations aimed at making the model robust. Data was fed in `ImageFolder` and was loaded using `DataLoader` (Kaptein et al., 2022). A pretrained ResNet18 was loaded with the ImageNet weights and the last fully-connected layer was changed to correspond to the number of the classes of shapes in the dataset. It was then transferred to a GPU (provided there was one available) and put in an evaluation mode to avoid the computation of gradients.

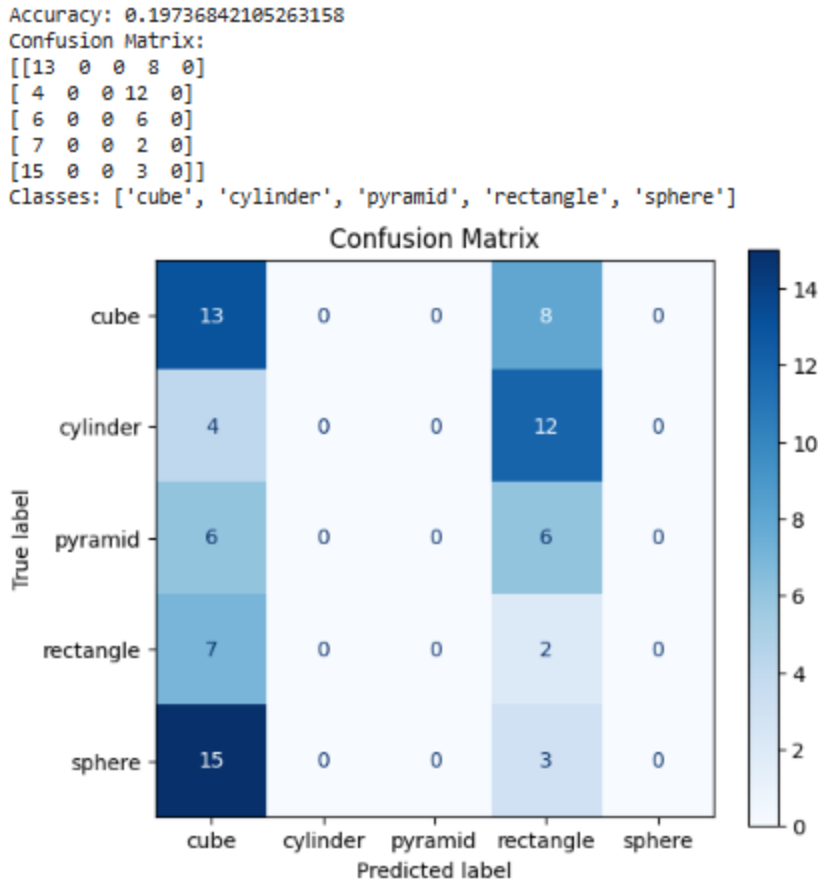


Figure 10: Performance matrix

The result of pretrained ResNet18 model, which is based on the overall accuracy of ~ 19.7 percent, shows that despite some success, the geometric shapes were not classified well. As shown in the confusion matrix, the vast majority of the predictions were not correctly classified, more so, on the side of the "rectangle" class (Kaptein et al., 2022). Such as 13 images of the cube and 15 images of the sphere were incorrectly classified as rectangle. Only a few classes, like the one that contained cylinder or that one which contained pyramid were predicted correctly, with all predictions across the board being zero in most of the shapes. The model was not trained to this end by using synthetic geometric shapes, so this poor performance is to be expected. The difference between the visual domain of the URDF-generated images and the natural images on which ResNet18 has been trained yields the evidence of the necessity of using the notion of the fine-tuning or even custom training to solve the domain-specific recognition problems.

Google Collab Link:

https://colab.research.google.com/drive/10u0_Vqm62_QjlcMPaq6AG5R3GY-IQUY7?usp=sharing

CHAPTER TWO: Vision-Guided Autonomous Navigation in ROS (TheConstruct.ai)

2.1: Objectives

- To design a robotic autonomous simulation in TheConstruct.ai with TurtleBot3 where the environment is structured and visual navigation is to be demonstrated.
- To set up on board RGB or RGB-D cameras in publishing image data in real time for object detection and recognition purposes.
- To detect an object (can use pretrained or OpenCV) according to color and/or marking on a cube or marker on the visual frame of the robot and determine the relative direction (left, center or right).
- To implement and realize vision-tracked navigation logic to make the robot match and approach the identified goal on the basis of its location inside the picture, as an emulation of real-life responsive motion.
- Unique Name of ROSject : **vision_nav_proj3knDlp4hi8**

2.2: Setup the Simulation Environment

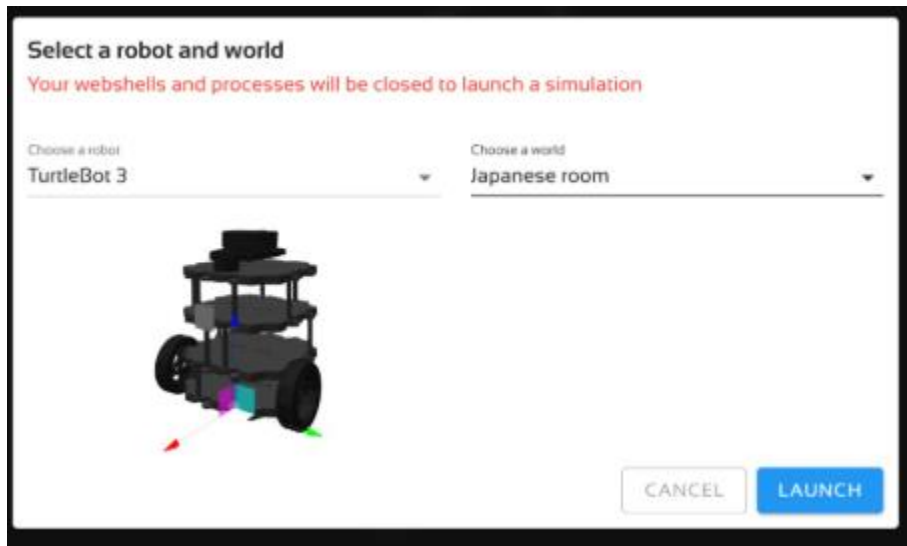


Figure 11: Setup robot

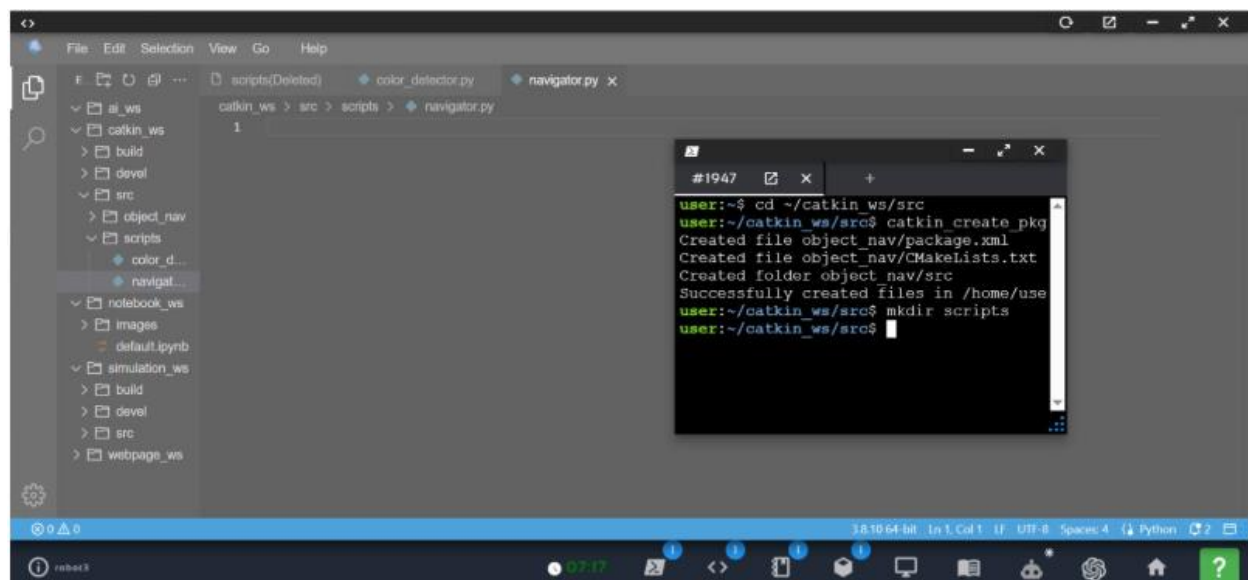


Figure 12: Creating directories

The construct.ai offers ROS simulation environment (cloud-based) to ease the implementation of TurtleBot3 in a pre-defined virtual room or a hall. The initial one is the launch of the TurtleBot 3 model in this space and the initiation of the robot there. The visual target of navigation is a colored cube or marker (Sathi G et al., 2024). To model such an unpredictable and hostile environment, this object will be located manually or arbitrarily in other places on the environment during every

trial. The positioning of targets is different and thereby facilitates the analysis of adaptability and real-world factors in this robot in visual navigation (Sathi G et al., 2024). The setting of the environment must be similar to allow reproducible testing and well-controlled experimentation.

2.3: Sensor Setup

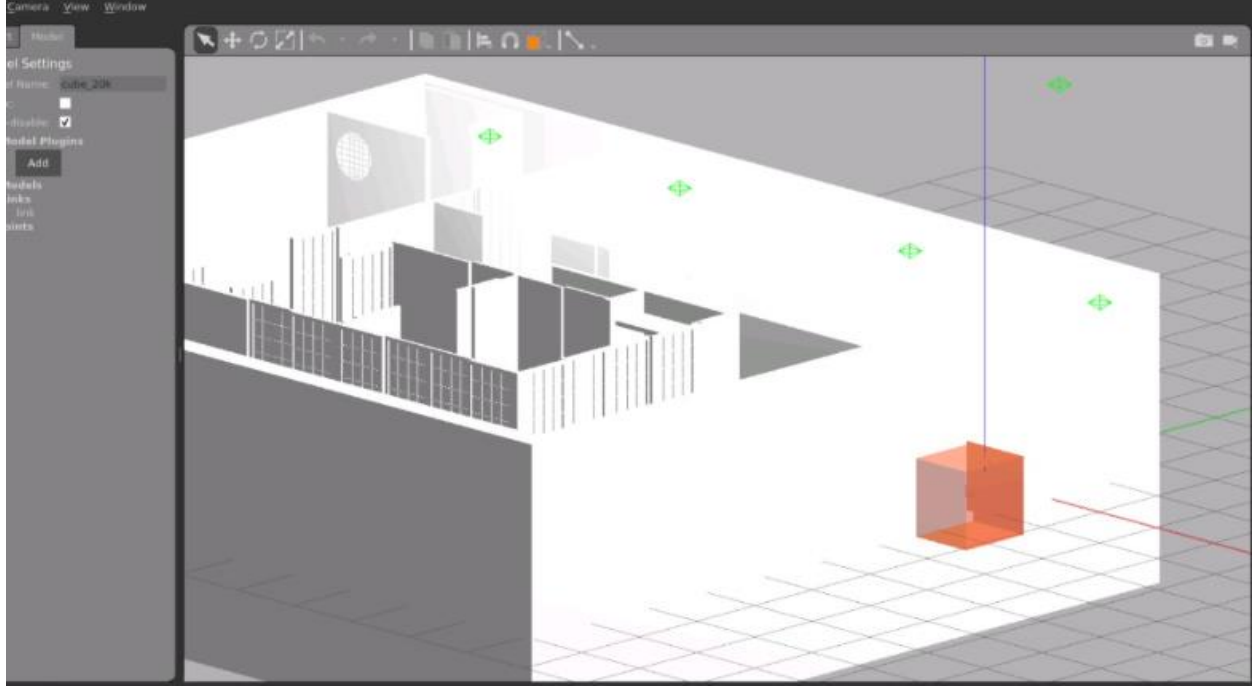


Figure 13: Colored cube setup

The TurtleBot3 has an RGB camera (or RGB-D) with most of them attached to the front. We need to configure correctly that the camera is actively broadcasting image data to a particular ROS topic, e.g. `/camera/rgb/image_raw`. This video data source is the main source of visual data which the robot will be able to observe to identify surroundings and objects of interest (Siriweera and Naruse, 2021). ROS node should be set to subscribe to this topic in order to process frames of real time images. Also, a calibrated RGB-D camera can facilitate perception due to excellent combination of color and depth ranging data that helps in object localization or instances of collision avoidance. It is important to make sure that the camera stream is available in real-time and start driving and detection logic (Siriweera and Naruse, 2021).

2.4: Object Detection

Depending on the use of OpenCV-based color filtering or a pretrained neural network, object detection is carried out. The less complex one is a target-isolating algorithm built on OpenCV

through the use of its distinct color features (e.g. HSV). This permits real-time target segmentation in controlled areas. Otherwise, the model trained on Task 1 could be reused to detect such objects as the cube or pyramids on the basis of their visual features. After detecting the object in the picture, whether it is on the left or center or right of the camera is found out by dividing the camera frame into left region or center or right (Song, Tanwani and Goldberg, 2025). This spatial data is used on navigation decisions. The detection module constantly monitors incoming frames to make sure that the robot is informed where the target is at all times even when the target has changed its position even the slightest or if it is partially covered up (Zhang, 2021).

2.5. Navigation Toward Target

```
user:~$ roslaunch object_nav ob
ject_nav.launch
... logging to /home/user/.ros/
log/e484192a-4cec-11f0-b532-024
2ac130007/roslaunch-1_xterm-109
32.log
Checking log directory for disk
usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usa
ge. Usage is <1GB.

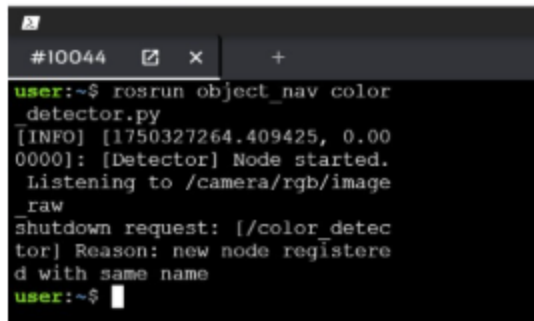
started roslaunch server http:/
/1_xterm:35795/

SUMMARY
=====

PARAMETERS
* /rostdistro: noetic
* /rosversion: 1.15.9

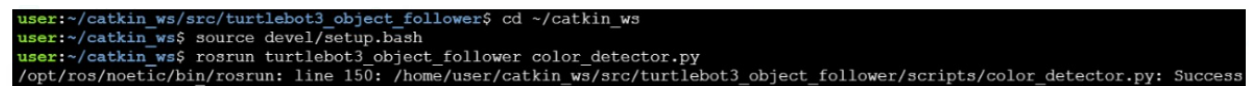
NODES
/
  color_detector (object_nav/
```

Figure 14: ROS launch



```
#10044 [x] [x] +
user:~$ rosrn object_nav color
detector.py
[INFO] [1750327264.409425, 0.00
0000]: [Detector] Node started.
Listening to /camera/rgb/image
raw
shutdown request: [/color_detec
tor] Reason: new node registere
d with same name
user:~$
```

Figure 15: Color detection command



```
user:~/catkin_ws/src/turtlebot3_object_follower$ cd ~/catkin_ws
user:~/catkin_ws$ source devel/setup.bash
user:~/catkin_ws$ rosrn turtlebot3_object_follower color_detector.py
/opt/ros/noetic/bin/rosrun: line 150: /home/user/catkin_ws/src/turtlebot3_object_follower/scripts/color_detector.py: Success
```

Figure 16: Detection success

The navigation reasoning is constructed on the position of the object in the picture frame. When the object is on the center, then the robot will follow the instruction to move forward as it assumes that it is on the same line as the target (Yu, Kamat and Menassa, 2024). On detecting the object to the left or to the right the robot makes a small angular rotation in that direction to re-center the object. This system allows the adjustment to be taken in tracking, in a smooth and incremental form. It is similar to visual servoing, in which motion control is directly effected by visual feedback. ROS cmd_vel topic is used to send the velocity commands in the robot to TurtleBot3 in the linear and angular forms (Zhang, 2021). This act will repeat until an object is extremely near the robot or until it finishes its assignment. This is an example of a reactive control system in which a visual pointer can be used to direct a robot in real time without physically recorded positions or other tracking tool.

CONCLUDING REMARKS

Integration of inventions on robotic vision and autonomous navigation was accomplished successfully in this project in two tasks of simulation. Task 1 consisted of a static setting constructed in PyBullet of custom URDF models comprising some geometric shapes. A synthetic dataset was generated by RGB images recorded at various camera angles. This dataset was used to test a pretrained model ResNet18 on shape classification applications and did explore the possibilities and shortcomings of applying store-bought model without fine-tuning. This was done in Task 2 with a new along with a vision-guided autonomous mobile robot implemented in TheConstruct.ai using ROS. Real-time image processing and directional control logic enabled the TurtleBot3 robot to identify a target object and navigate towards the object. Collectively, these operations demonstrated fundamental concepts of computer vision, sensor fusion and autonomous reasoning. The project also highlighted the significance of cloud based tools to perform simulation so that repeatable and scalable testing can be done. On the whole, the work is a mix between theory and practicality and provides the basic knowledge behind current robotic vision systems.

BIBLIOGRAPHY

Albert, J. and Wong, Y.-H. (2024). Adaptive Robotics: Integrating Robotic Simulation, AI, Image Analysis, and Cloud-Based Digital Twin Simulation for Dynamic Task Completion. *Lecture notes in computer science*, pp.262–271. doi: https://doi.org/10.1007/978-3-031-60615-1_17.

Baig, M.A., Han, L., Yang, D., Danaish, Dong, G., Niaz, M.M., Sibtain, M., Hassan, A.S.M., Rabeh, A.S.S. and Al-Shameri, M.A.A. (2024). Cloud-Based Smart Coordinated Robot System for Strawberry Harvesting in Controlled Greenhouse Environment. *2024 International Conference on Advanced Mechatronic Systems (ICAMEchS)*, [online] pp.219–224. doi: <https://doi.org/10.1109/icamechs63130.2024.10818802>.

Dawarka, V. and Bekaroo, G. (2022). Building and evaluating cloud robotic systems: A systematic review. *Robotics and Computer-Integrated Manufacturing*, 73, p.102240. doi: <https://doi.org/10.1016/j.rcim.2021.102240>.

Fabeha Raheel, Laibah Raheel, Mehmood, H., Kadri, M.B. and Khan, U.S. (2024). Internet of Robots: An Open-source Framework for Cloud-Based Global Remote Control and Monitoring of Robots. [online] pp.1–6. doi: <https://doi.org/10.1109/icrai62391.2024.10894149>.

Harinakshi, C., Masooda, A. and Suchetha, G. (2024). Cloud Infrastructure for Robotics. *Advances in Computational Intelligence and Robotics*, [online] pp.20–37. doi: <https://doi.org/10.4018/979-8-3693-1914-7.ch002>.

Kaptein, F., Kiefer, B., Cully, A., Celiktutan, O., Bierman, B., Rijgersberg-peters, R., Broekens, J., Van Vught, W., Van Bekkum, M., Demiris, Y. and Neerincx, M.A. (2022). A Cloud-based Robot System for Long-term Interaction: Principles, Implementation, Lessons Learned. *ACM Transactions on Human-Robot Interaction*, 11(1), pp.1–27. doi: <https://doi.org/10.1145/3481585>.

Saini, M., Sharma, K. and Rajesh Doriya (2022). An empirical analysis of cloud based robotics: challenges and applications. *International Journal of Information Technology*, 14(2), pp.801–810. doi: <https://doi.org/10.1007/s41870-021-00842-4>.

Sathi G, Varshney, N., Sharma, P., B.Jasmine Punitha, Sundar, R. and SubbaRao, S.P.V. (2024). A Development of Cloud Based Robotics Design Networks for Industry Applications. [online] pp.320–325. doi: <https://doi.org/10.1109/icacite60783.2024.10616950>.

Siriweera, A. and Naruse, K. (2021). Survey on Cloud Robotics Architecture and Model-Driven Reference Architecture for Decentralized Multicloud Heterogeneous-Robotics Platform. *IEEE Access*, 9, pp.40521–40539. doi: <https://doi.org/10.1109/access.2021.3064192>.

Song, D., Tanwani, A.K. and Goldberg, K. (2025). Networked, Cloud and Fog Robotics. *Robotics Goes MOOC*, pp.271–313. doi: https://doi.org/10.1007/978-3-319-77270-7_8.

Yu, H., Kamat, V.R. and Menassa, C.C. (2024). Cloud-Based Hierarchical Imitation Learning for Scalable Transfer of Construction Skills from Human Workers to Assisting Robots. *Journal of Computing in Civil Engineering*, 38(4). doi: <https://doi.org/10.1061/jccee5.cpeng-5731>.

Zhang, N. (2021). A Cloud-Based Platform for Big Data-Driven CPS Modeling of Robots. *IEEE Access*, 9, pp.34667–34680. doi: <https://doi.org/10.1109/access.2021.3061477>.

Zhumaniyaz Mamatnabiyev, Chronis, C., Iraklis Varlamis, Yassine Himeur and Meirambek Zhaparov (2024). A Holistic Approach to Use Educational Robots for Supporting Computer Science Courses. *Computers*, 13(4), pp.102–102. doi: <https://doi.org/10.3390/computers13040102>.

APPENDIX

```
# Cube URDF
with open("urdf_models/cube.urdf", "w") as f:
    f.write('''<?xml version="1.0"?><robot name="cube">
    <link name="cube_link">
        <visual>
            <geometry><box size="0.3 0.3 0.3"/></geometry>
            <origin xyz="0 0 0.15"/>
            <material name="red"><color rgba="1 0 0 1"/></material>
        </visual>
    </link>
</robot>''')
```

```
# Sphere URDF
with open("urdf_models/sphere.urdf", "w") as f:
    f.write('''<?xml version="1.0"?><robot name="sphere">
    <link name="sphere_link">
        <visual>
            <geometry><sphere radius="0.15"/></geometry>
            <origin xyz="0 0 0.15"/>
            <material name="blue"><color rgba="0 0 1 1"/></material>
        </visual>
    </link>
</robot>''')
```

```
# Cylinder URDF
with open("urdf_models/cylinder.urdf", "w") as f:
    f.write('''<?xml version="1.0"?><robot name="cylinder">
    <link name="cylinder_link">
        <visual>
            <geometry><cylinder radius="0.15" length="0.3"/></geometry>
            <origin xyz="0 0 0.15"/>
            <material name="green"><color rgba="0 1 0 1"/></material>
        </visual>
    </link>
</robot>''')
```

```
# Rectangle URDF
with open("urdf_models/rectangle.urdf", "w") as f:
    f.write('''<?xml version="1.0"?><robot name="rectangle">
    <link name="rectangle_link">
        <visual>
            <geometry><box size="0.6 0.3 0.2"/></geometry>
            <origin xyz="0 0 0.1"/>
            <material name="purple"><color rgba="0.6 0.2 0.8 1"/></material>
        </visual>
    </link>
</robot>''')
```

```
# STEP 4: Load into PyBullet
p.connect(p.DIRECT)
p.setAdditionalSearchPath(pybullet_data.getDataPath())
p.loadURDF("plane.urdf")

shapes = [
    "cube.urdf", "cube.urdf",
    "cylinder.urdf", "cylinder.urdf",
    "sphere.urdf", "sphere.urdf",
    "pyramid.urdf",
    "rectangle.urdf"
]
positions = [
    [0, 0, 0.1], [1, 0, 0.1],
    [-1, 0, 0.1], [0, 1, 0.1],
    [1, 1, 0.1], [-1, 1, 0.1],
    [0, -1, 0.1],
    [1, -1, 0.1]
]

for shape, pos in zip(shapes, positions):
    p.loadURDF(f"urdf_models/{shape}", basePosition=pos)

for _ in range(240):
    p.stepSimulation()
    time.sleep(1/240)
```

```

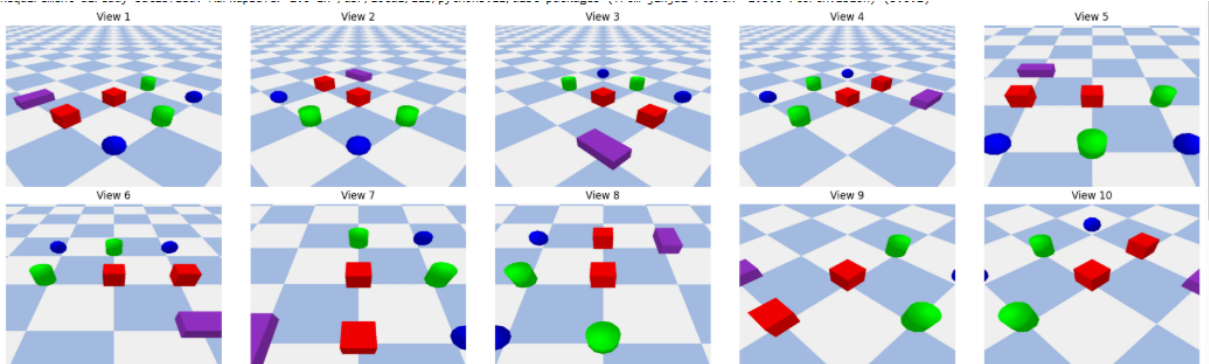
# STEP 5: Capture Camera Views
os.makedirs("dataset_views", exist_ok=True)
camera_positions = [
    [2, 2, 2], [-2, 2, 2], [2, -2, 2], [-2, -2, 2],
    [0, 2, 2], [0, -2, 2], [1.5, 0, 2], [-1.5, 0, 2],
    [1, 1, 2], [-1, -1, 2]
]

for i, pos in enumerate(camera_positions):
    view_matrix = p.computeViewMatrix(pos, [0, 0, 0], [0, 0, 1])
    proj_matrix = p.computeProjectionMatrixFOV(60, 1, 0.1, 100)
    _, _, rgb, _, _ = p.getCameraImage(640, 480, view_matrix, proj_matrix)
    Image.fromarray(rgb).save(f"dataset_views/view_{i+1}.png")

p.disconnect()

# STEP 6: Display Views
fig, axs = plt.subplots(2, 5, figsize=(20, 6))
for i in range(10):
    img = Image.open(f"dataset_views/view_{i+1}.png")
    axs[i // 5][i % 5].imshow(img)
    axs[i // 5][i % 5].axis('off')
    axs[i // 5][i % 5].set_title(f"View {i+1}")
plt.tight_layout()
plt.show()

```



```

# STEP 7: Organize Dataset Folders
classes = ["cube", "cylinder", "sphere", "pyramid", "rectangle"]
for c in classes:
    os.makedirs(f"dataset/{c}", exist_ok=True)

# Assign one image per class
shutil.copy("dataset_views/view_1.png", "dataset/cube/img1.png")
shutil.copy("dataset_views/view_2.png", "dataset/cube/img2.png")
shutil.copy("dataset_views/view_3.png", "dataset/cylinder/img3.png")
shutil.copy("dataset_views/view_4.png", "dataset/cylinder/img4.png")
shutil.copy("dataset_views/view_5.png", "dataset/sphere/img5.png")
shutil.copy("dataset_views/view_6.png", "dataset/sphere/img6.png")
shutil.copy("dataset_views/view_7.png", "dataset/pyramid/img7.png")
shutil.copy("dataset_views/view_8.png", "dataset/rectangle/img8.png")

# STEP 7B: Safely Duplicate Images
for i in range(1, 6):
    shutil.copy("dataset/cube/img1.png", f"dataset/cube/img_dup_{i}.png")
    shutil.copy("dataset/cube/img2.png", f"dataset/cube/img_dup_b{i}.png")
    shutil.copy("dataset/cylinder/img3.png", f"dataset/cylinder/img_dup_{i}.png")
    shutil.copy("dataset/cylinder/img4.png", f"dataset/cylinder/img_dup_b{i}.png")
    shutil.copy("dataset/sphere/img5.png", f"dataset/sphere/img_dup_{i}.png")
    shutil.copy("dataset/sphere/img6.png", f"dataset/sphere/img_dup_b{i}.png")
    shutil.copy("dataset/pyramid/img7.png", f"dataset/pyramid/img_dup_{i}.png")
    shutil.copy("dataset/rectangle/img8.png", f"dataset/rectangle/img_dup_{i}.png")

```

```

# STEP 8: Preprocessing with Augmentation
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])
dataset = ImageFolder("dataset", transform=transform)
dataloader = DataLoader(dataset, batch_size=8, shuffle=False)

# STEP 9: ResNet18 Inference
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = models.resnet18(pretrained=True)
model.fc = nn.Linear(model.fc.in_features, len(dataset.classes))
model = model.to(device)
model.eval()

preds, targets = [], []
with torch.no_grad():
    for inputs, labels in dataloader:
        outputs = model(inputs.to(device))
        _, predicted = torch.max(outputs, 1)
        preds.extend(predicted.cpu().numpy())
        targets.extend(labels.numpy())

```

Accuracy: 0.19736842105263158

Confusion Matrix:

```
[[13  0  0  8  0]
```

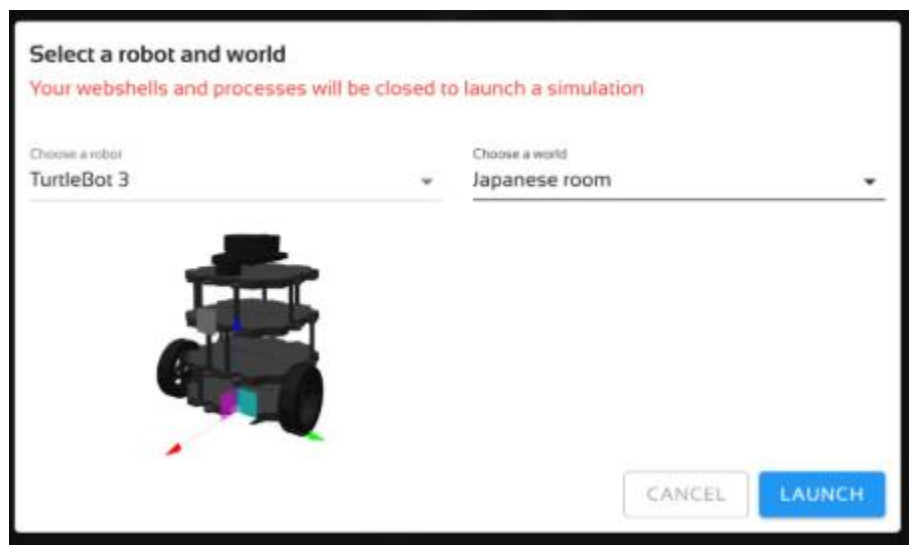
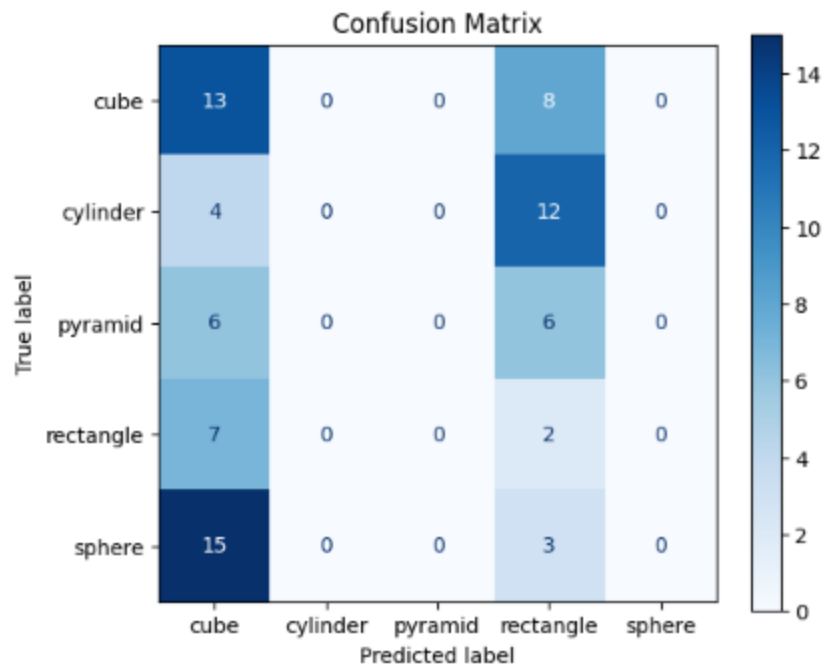
```
[ 4  0  0 12  0]
```

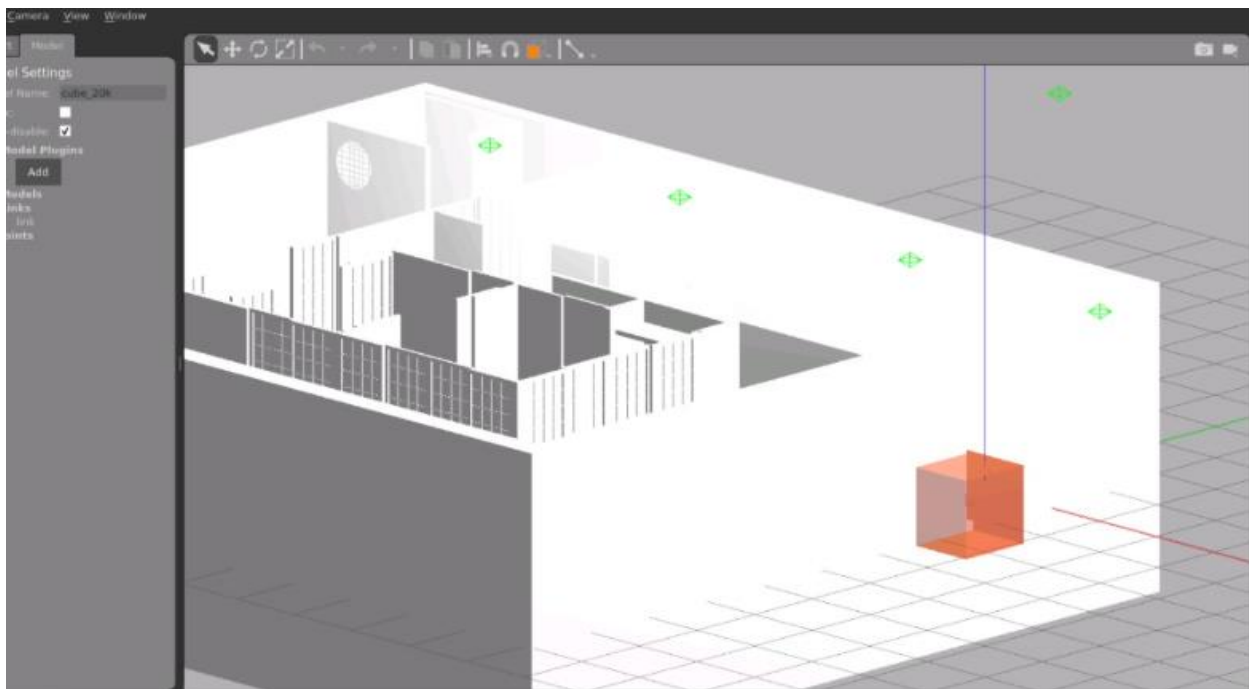
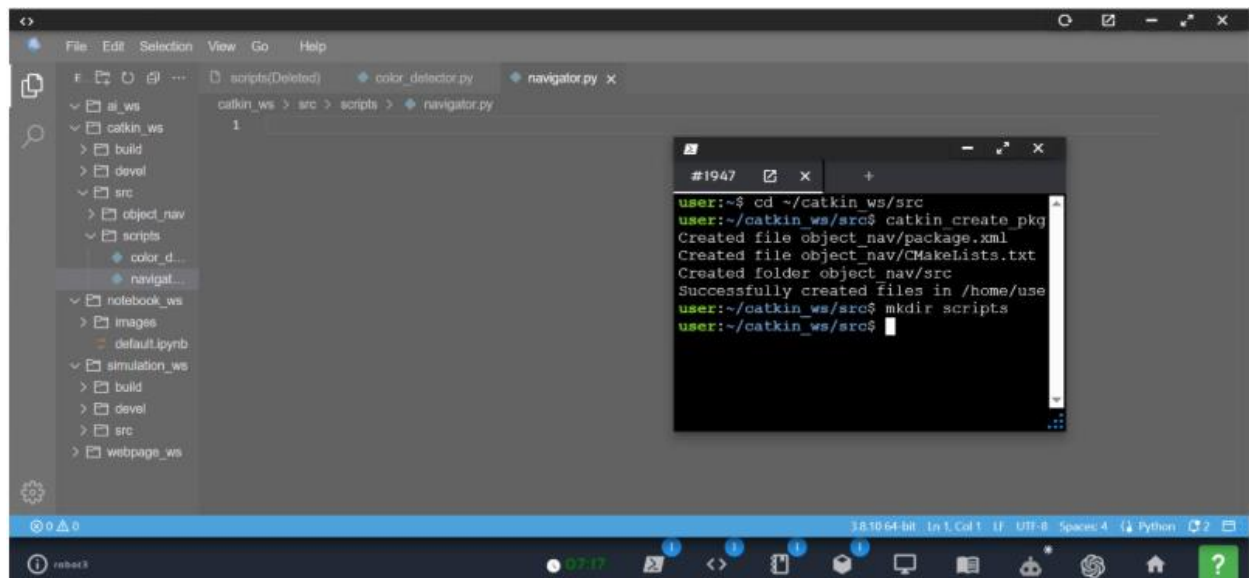
```
[ 6  0  0  6  0]
```

```
[ 7  0  0  2  0]
```

```
[15  0  0  3  0]]
```

Classes: ['cube', 'cylinder', 'pyramid', 'rectangle', 'sphere']





```

user:~$ roslaunch object_nav ob
ject_nav.launch
... logging to /home/user/.ros/
log/e484192a-4cec-11f0-b532-024
2ac130007/roslaunch-1_xterm-109
32.log
Checking log directory for disk
usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usa
ge. Usage is <1GB.

started roslaunch server http:/
/1_xterm:35795/

SUMMARY
=====

PARAMETERS
* /rostdistro: noetic
* /rosversion: 1.15.9

NODES
/
  color_detector (object_nav/

```

```

#10044 [x] +
user:~$ rosrund object_nav color
_detector.py
[INFO] [1750327264.409425, 0.00
0000]: [Detector] Node started.
Listening to /camera/rgb/image
_raw
shutdown request: [/color_detec
tor] Reason: new node registere
d with same name
user:~$

```

```

user:~/catkin_ws/src/turtlebot3_object_follower$ cd ~/catkin_ws
user:~/catkin_ws$ source devel/setup.bash
user:~/catkin_ws$ rosrund turtlebot3_object_follower color_detector.py
/opt/ros/noetic/bin/rosrun: line 150: /home/user/catkin_ws/src/turtlebot3_object_follower/scripts/color_detector.py: Success

```